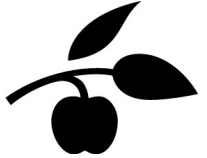


Model-Driven Software Development

Advanced topics on parsing

SDU 

Miguel Campusano

SDU Software Engineering

Previous class

Xtext and Code Generation

Summary

Xtext grammar
Code generation
Xtext - Xtend

Next:
Refine Class Description DSL

```
for(Entity e: system.getEntities()) {  
    generateFile(e.name+".java",  
        '''package university;  
import java.util.*  
public class «e.name» {  
    «FOR a: e.attributes»  
    private «a.type.javaType» «a.name»;  
    . . .  
    '''  
}
```

The logo for Xtend, featuring a stylized blue 'X' followed by the word 'tend' in a dark grey sans-serif font.

Code Generation

In this course: DSL that generates source code

Internal/External DSL creates a metamodel instance

From metamodel instance, code is generated

Code can be executed by an external agent

External DSL: recommend Xtext (syntax) and *Xtend* (code generation)

Code Generation: Xtend

Metamodel
Instance



```
for(Entity e: system.getEntities()) {  
    generateFile(e.name+".java",  
        '''package university;  
import java.util.*  
public class «e.name» {  
    «FOR a: e.attributes»  
    private «a.type.javaType» «a.name»;  
    . . .  
    '''  
}
```

```
package university;  
import java.util.ArrayList;  
public class Student extends Person {  
    private int id;  
    private List<Course> courses = new ArrayList<>();  
    public Student(int id, String name, int age) {  
        super(name, age);  
        this.setId(id);  
    }  
    public int getId() {  
        return id;  
    }  
    public void setId(int id) {  
        this.id = id;  
    }  
    public List<Course> courses() {  
        return courses;  
    }  
    public void addCourse(Course course) {  
        courses.add(course);  
    }  
}
```

Implementation notes

- Write grammar, refine, check metamodel, etc

 - For grammar: use 2 or 3 examples

- Use existing code as a starting point

 - Insert into template

 - Incrementally add Xtext annotations

- Overall structure: template approach

- Specific code: extract from metamodel (transformer)

- Keep templates as simple as possible (use helper functions)

- Exploit Xtend features (type-based switch, etc...)

~~Discussion~~

Mid-term evaluation

<https://pollev.com/mica>

Mid-term evaluation

What is it NOT working?

What is it working (and should continue doing)?

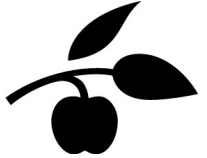
Extra comments

Me / Lectures / Discussions / Tools / Polls / Assignments / Slides / in-class exercises / Discussion board for exercises / Project work / ...

PolleEv.com/mica

Model-Driven Software Development

Advanced topics on parsing

SDU 

Miguel Campusano

SDU Software Engineering

Today

Abstract Syntax Tree

Parsing problems

Motivation

Math Expressions

Class Description V1

system University

entity Person

attribute name **is** String

attribute age **is** Number

entity Student

 - id **is** Number

entity Teacher

entity Course

attribute title **is** String

Student **is** Person

Teacher -> Person

Teacher **1--1** Course

Student ***--*** Course

S ::= 'system' ID E* R*

E ::= 'entity' ID At*

At ::= ('attribute' | '-') ID 'is' T

T ::= ...

R ::= I|C

I ::= ID_E ('is' | '->') ID_E

C ::= ID_E ('1-1' | '*-*') ID_E

Class Description V2: math expressions

system University

entity Person

attribute name **is** String

attribute age **is** Number

require age >= 0

entity Student

 - id **is** Number

require age >= 15

entity Teacher

entity Course

attribute title **is** String

attribute ects **is** Number

require ects > 0 && ects <= 3*10

Student **is** Person

Teacher -> Person

Teacher **1--1** Course

Student ***--*** Course

S ::= 'system' ID E* R*

E ::= 'entity' ID (At|Q)*

At ::= ('attribute' | '-') ID 'is' T

Q ::= 'require' Exp

T ::= ...

R ::= I|C

I ::= ID_E ('is' | '->') ID_E

C ::= ID_E ('1-1' | '*-*') ID_E

Exp ::= ...

Class Description V2: math expressions

var comp exp

ects $\leq 3 * 10$

var $> 10 + 5$

var $== 10 + 5 * 3$

var $== (10 + 5) * 3$

exp comp var?

exp comp exp?

Abstract Syntax Tree

AST: Abstract Syntax Tree

Abstract syntactic structure of the parsed program

Tree data-structure

In Xtext

The grammar defines the metamodel -> defines AST

Language theory: CFG -> parse tree -> AST

AST: Abstract Syntax Tree

...

Exp: Number | Plus

Plus: left=Exp '+' right=Exp

Number: value=INT

10 + 5 + 3

Activity

Define Math expression AST

AST: in-class activity

system University

entity Person

attribute name **is** String

attribute age **is** Number

entity Student

 - id **is** Number

entity Teacher

entity Course

attribute title **is** String

Student **is** Person

Teacher -> Person

Teacher **1--1** Course

Student ***--*** Course

Activity

Define AST

Upload to discussion board

Get the right AST
Using parsing strategies

Parsing problems

We want a well-defined AST

It simplifies the generation (interpretation) of our code

We give some *semantics* to the AST

Ambiguity

Left recursive grammars

Associativity

Precedence

Parsing problem #1

Ambiguity

Parsing problem: Ambiguity

...

Exp: Number | Plus

Plus: left=Exp '+' right=Exp

Number: value=INT

10 + 5 + 3

Parsing problem: Ambiguity

...

Exp: Number | Plus

Plus: left=Exp '+' right=Exp

Number: value=INT

10 + 5 + 3

...

Exp: Number | Plus

Plus: left=Exp ('+' **right=Number**)?

Number: value=INT

Parsing problem #2

Left recursive grammar

Parsing problem: Left recursive grammar

...

Exp: Number | Plus

Plus: left=**Exp** ('+' right=Number)?

Number: value=INT

10 + 5 + 3

Parsing problem: Left recursive grammar

...

Exp: Number | Plus

Plus: left=**Exp** ('+' right=Number)?

Number: value=INT

10 + 5 + 3

...

Exp: ~~Number~~ | Plus

Plus: left=**Number** ('+' right=**Exp**)?

Number: value=INT

Parsing problem: rewriting AST

...

Exp: Plus

Plus: left=Number ('+' right=Exp)?

Number: value=INT

10

Parsing problem: rewriting AST

```
...  
Exp: Plus  
Plus: left=Number ('+' right=Exp)?  
Number: value=INT
```

10

```
...  
Exp: Number ({Plus.left=current} '+' right=Exp)?  
Number: value=INT
```

Parsing problem #3

Associativity

Parsing problem: Associativity

...

Exp: Number ({Plus.left=current} '+' right=Exp)?

Number: value=INT

10 + 5 + 2

10 - 5 - 2

Semantic in AST

Abstract *syntactic* structure of the parsed program

We give a meaning (semantic) to the AST

Parsing problem: Associativity

...

Exp: Number ({Plus.left=current} '+' right=Exp)?

Number: value=INT

10 + 5 + 2

10 - 5 - 2

...

Exp: Number ({Plus.left=current} '+' right=**Number**)*

Number: value=INT

Parsing problem: Associativity

...

Exp: Number ('+' Number)*

Number: value=INT

10 + 5 + 2

10 - 5 - 2

...

Exp: Number ({Plus.left=current} '+' right=**Number**)*

Number: value=INT

Parsing problem #4

Precedence

Parsing problem: Precedence

...

Exp: Number (({Plus.left=current} '+' | {Mult.left=current} '*') right=Number)*

Number: value=INT

10 + 5 * 2

Parsing problem: Precedence

...

Exp: Number (({Plus.left=current} '+' | {Mult.left=current} '*') right=Number)*
Number: value=INT

10 + 5 * 2

Multiplication has a 'higher level' of precedence than addition

Intuition:

Plus is an addition of *terms*

Term is a multiplication of *factors*

Factor is a number

Parsing problem: Precedence

Multiplication has a 'higher level' of precedence than addition

Intuition:

Plus is an addition of *terms*

Term is a multiplication of *factors*

Factor is a number

```
...
Exp: Number (
    ({Plus.left=current} '+' |
     {Mult.left=current} '*')
    right=Number)*
Number: value=INT
```

```
...
Exp: Plus
Plus returns Expression:
    Term ({Plus.left=current} '+' right=Term)*
Term returns Expression:
    Factor ({Mult.left=current} '*'
    right=Factor)*
Factor returns Expression: Number
Number: value=INT
```

Parsing problem: Precedence

```
...  
Exp: Plus  
Plus returns Expression: Term ({Plus.left=current} '+' right=Term)*  
Term returns Expression: Factor ({Mult.left = current} '*' right=Factor)*  
Factor returns Expression: Number  
Number: value=INT
```

10 + 5 * 3 + 1

(10 + 5) * (3 + 1)

Parsing problem: Precedence

```
...  
Exp: Plus  
Plus returns Expression: Term ({Plus.left=current} '+' right=Term)*  
Term returns Expression: Factor ({Mult.left = current} '*' right=Factor)*  
Factor returns Expression: Number | '(' Exp ')'  
Number: value=INT
```

10 + 5 * 3 + 1

(10 + 5) * (3 + 1)

Parsing problem summary

That was a lot

Parsing problem: summary

Exp ::= INT | Exp '+' Exp Ambiguous

Exp ::= INT | INT '+' Exp Right Associative

Exp ::= INT | Exp '+' INT Left associative, but left recursive grammar

Exp ::= INT ('+' INT)* Non-associative

Parsing problem: summary

$\text{Exp} ::= \text{INT} \mid \text{Exp} \text{ '+' } \text{Exp}$ Ambiguous

$\text{Exp} ::= \text{INT} \mid \text{INT} \text{ '+' } \text{Exp}$ Right Associative

$\text{Exp} ::= \text{INT} \mid \text{Exp} \text{ '+' } \text{INT}$ Left associative, but left recursive grammar

$\text{Exp} ::= \text{INT} (\text{ '+' } \text{INT})^*$ Left associative with tree rewrite action ($\{ \text{.} = \text{current} \}$)

Group project

Project: expectations

Group

Metamodel design → Solve a family of problems (topic)

Internal DSL → Good technique in a PL of your choice

External DSL syntax → Translate your internal DSL to external DSL

Code generation → Generate working code

External DSL advance → a somewhat interesting grammar

Left recursion → e.g., math exp

Name resolution → use of square brackets

Project: expectations

Individual extension

More *interesting* features → something *new* to your DSL

Anything left from the rest of the course → validation, scope, px, etc

Type Checker → complex validation

Modify default scope → *example next week*

Visual DSL → or something more than “text”

Testing → complex enough (not covered but enough information in the book)

Module system → *import* feature

...

Summary

Summary

Abstract Syntax Tree

Parsing problems

Next:

Validation

Scope

$\text{Exp} ::= \text{INT} \mid \text{Exp} \text{ '+' } \text{Exp}$ Ambiguous

$\text{Exp} ::= \text{INT} \mid \text{INT} \text{ '+' } \text{Exp}$ Right Associative

$\text{Exp} ::= \text{INT} \mid \text{Exp} \text{ '+' } \text{INT}$ Left associative, but left recursive grammar

$\text{Exp} ::= \text{INT} \text{ ('+' INT)}^*$ Left associative with tree rewrite action

Group Project Work

Week 6 (same as week 5)

Example programs:

DSL

Output program

Code generator in Xtend

Re-design your DSL to support a more interesting grammar

Name resolution → Square Brackets

Left recursion → e.g., math exp